2026 Spring

- https://github.com/cqframework/CQL-Formatting-and-Usage-Wiki/wiki/Authoring-Measures-in-CQL
- Clinical Quality Language (CQL)
- FHIR (hl7.org)
- https://hl7.org/fhirpath/

Meeting Notes (17/01/2026):
- https://en.wikipedia.org/wiki/Lumpers_and_splitters
- Snomed LOINC
- ICD 10
- Value Sets
- Vectorization of value sets

Meeting Notes:

- ECQMs: how providers are providing quality control
- ECQMs for osha compliance
- Medical surveillance
- Have the AI take Osha regulations and build ecqms
- CQL -> language like sql
- Load cql into an ehr and check which patient is non-compliant
- Team working on this, can contribute
- RFP Request for press release
- Quality metrics
- Write the press release
- Discussions about ecqms, osha requirements, instances and examples

V1.1 Proposal (Updated)

WorkWell Measure Studio (OSHA-to-eCQM/CQL → measure runs → actionable compliance worklist)

Problem and Proposal | Taleef Tamsal

Restating the Problem:

Occupational health and compliance teams are accountable for medical surveillance requirements (annual physicals by role, TB screening, immunizations, hearing conservation, respirator clearance, HAZWOPER) that vary by site, job role, and exposure risk. In practice, these requirements are operationalized through spreadsheets, manual chart review, and ad hoc reporting. The result is predictable: requirements are interpreted inconsistently, “due vs overdue” logic differs across teams, and work becomes reactive rather than systematic

The core issue is not a lack of dashboards, it is the absence of a repeatable translation layer from policy/OSHA requirements to executable, testable compliance logic and a reliable operational workflow that turns results into accountable follow-up actions. If compliance determinations are not explainable (why someone is flagged) and not auditable (who changed what, when, and why), then teams cannot trust the system, and leadership cannot defend decisions during audits.

Proposed Solution:

Working backwards: (See the section below this proposal, starting from the press release)

I propose WorkWell Measure Studio: a policy-to-CQL authoring and execution workflow that converts OSHA/internal surveillance requirements into versioned eCQMs (CQL), executes them against occupational health data, and operationalizes outcomes as an actionable worklist of cases.

This proposal follows the working-backwards method using the required artifacts:

1. **Press release (working backwards artifact)**
   Enterprise Health Announces WorkWell Measure Studio: Turning OSHA Requirements into Executable eCQMs and CQL with an actionable worklist for Medical Surveillance (Can be found right below the current proposal)

2. **Wireframe/storyboard + short (working backwards artifact)**
   a. Storyboard / Prototype: https://v0-work-well-measure-studio.vercel.app/

   b. YouTube Short (walkthrough): https://youtube.com/shorts/ojDNq38NksA?feature=share

   c. Discussion with ChatGPT in formulating this proposal (ignore latest chats): https://chatgpt.com/share/696afe4e-67e8-800c-9c23-1aa4a7927ff5
   d. V0 Chat: https://v0.app/chat/work-well-measure-studio-jVpzy2tuZyn?ref=WUIG21

3. **Workflow (system view)**
   (PlantUML workflow diagram images can be found in the section below the proposal)

How the solution works (end-to-end flow):

A) Authoring

- A measure author creates a measure and its version (Draft → Approved → Active).
- The Spec tab captures structured requirements: eligibility, exclusions/waivers, compliance window, required data elements, and policy references.
- The CQL tab contains executable logic (FHIR R4), which must compile/validate before release.
- Value sets and dependencies are attached and visible.
- Tests (fixtures + expected outcomes) block release if failing.

B) Execution

- A user triggers a manual run (and later a scheduled run) over a defined scope (program/site/measure).
- The run persists a run summary, logs, and per-employee outcomes.
- Outcomes include Compliant, Due Soon, Overdue, Missing Data, Excluded.

C) Operations

- Non-compliant outcomes upsert cases into a worklist (idempotent key: employee + measure version + evaluation period) to prevent duplicates on reruns.
- Case managers can open a case and see:
    - Why flagged: evidence payload showing the exact inputs that drove the result (role/site eligibility, last qualifying exam date, window calculation, waiver status).
    - Evidence timeline and at least one operational action (outreach/scheduling/evidence metadata).

    - Rerun-to-verify: rerun and confirm closure, with an audit trail.

D) Auditability

- Audit events capture measure edits, approvals/releases, run start/end, case lifecycle changes, and notifications/actions.

Technologies:

The proposal is intentionally implementable in an Enterprise Health-style environment while remaining demo-friendly.

Frontend (prototype and eventual UI)

- React / Next.js style UI (as represented in storyboard)
- Component library patterns suitable for enterprise dashboards (tables, filters, status chips, drill-down detail pages)

Backend services (MVP-level)

- Measure service: Measure + MeasureVersion models, lifecycle states, ownership, policy references, tags
- Compile/validate service: backend endpoint for CQL validation (returns errors/warnings, dependency resolution status)
- Run service: execution orchestration to evaluate Active measures against data and persist outcomes + run logs
- Case service: deterministic upsert of cases from outcomes, case lifecycle + action logging
- Audit/event log: append-only audit schema linking run IDs, case IDs, measure versions

Data model / interoperability

- FHIR R4 alignment for clinical/occupational health observations where applicable

- Value set registry: identifiers, names, versions, and resolvability checks at compile/release time

- Demo-grade “connected” flow: HRIS roster + basic FHIR resources + optional IH exposure feed; fallback seeded synthetic data for deterministic demos

AI + MCP (planned integration layer)

- AI assists authoring and operations (draft spec, summarize runs, generate outreach drafts, explain “why flagged”) but does not decide compliance.

- MCP can expose WorkWell capabilities as typed tools for an agent (read tools + gated write tools) once the deterministic pipeline is stable.

Anticipated Challenges:

1. **Data completeness and mapping**

    - Real-world occupational health data often has missing fields, inconsistent codes, and stale timestamps.
      Mitigation: treat Missing Data as a first-class outcome, show data freshness/missingness in run summary, avoid misclassifying missing data as overdue.

2. **Value set drift and code mismatch**

    - Measures depend on stable coding systems and resolvable value sets.
      Mitigation: enforce value set/dependency visibility, require resolvable value sets at compile/release time, include compile-time warnings and basic runtime checks.

3. **Over-alerting and worklist fatigue**

    - Too many flags overwhelm teams and reduce adoption.
      Mitigation: prioritize outcomes (Due Soon vs Overdue vs Missing Data), provide filters by program/site/role, support templated notification workflows, and emphasize idempotent reruns over repeated noise.

4. **Audit and explainability expectations**

    - Without defensible “why flagged” evidence, the system becomes hard to trust.
      Mitigation: store structured evidence payloads per outcome, link every case to

run ID and measure version, maintain append-only audit events.

5. **Execution reliability**

    - Runs can be long-running or partially failing.
      Mitigation: persist partial logs and summaries, support resumable runs, keep prior outcomes stable until replacement outcomes are written.

Timeline:

This is a rough progression as the stage names reflect WorkWell's needs and the working-backwards flow.

Stage 1: Research and requirements grounding (1–2 weeks)

- Confirm 3–4 initial OSHA-aligned programs/measures (e.g., Hearing Conservation audiogram cadence, TB Surveillance, annual physical by role, flu vaccination compliance by role).
- Define minimum FHIR resource set and minimum roster fields for the demo.
- Establish outcome taxonomy and case lifecycle.

Stage 2: Authoring backbone (2–3 weeks)

- Measure Catalog + Versioning
- Spec tab (structured requirements)
- CQL editor + compile/validate gate
- Minimal value set linkage visibility

Stage 3: Run pipeline + outcomes (2–3 weeks)

- Manual run orchestrator + run logs + run summary
- Persist per-employee outcomes + evidence payload

- Deterministic rerun behavior

Stage 4: Worklist operations + verification loop (2–3 weeks)

- Case upsert engine (idempotent)
- Case detail (why flagged + evidence timeline)
- One action end-to-end (outreach/scheduling/evidence metadata)
- Rerun-to-verify and closure behavior

Stage 5: Demo hardening and documentation (1–2 weeks)

- Seeded demo dataset + deterministic test cases
- Exportable run logs/audit trail view for credibility
- Documentation mapping to Enterprise Health workflows and future MCP/AI integration

Optional extension: Ozwell/AI + MCP integration (post-MVP or parallel, as time permits)

- Read-only MCP tools + “summarize/explain” assistant features
- Then gated write tools for drafts (spec/outreach) and rerun verification

Youtube Short: https://youtube.com/shorts/ojDNq38NksA?feature=share

Storyboard: https://v0-work-well-measure-studio.vercel.app/

Press Release

Enterprise Health Announces **WorkWell Measure Studio**: Turning OSHA Requirements into Executable eCQMs and CQL with an actionable worklist for Medical Surveillance

05/01/2026, Fort Wayne, IN Enterprise Health today announced WorkWell Measure Studio, a new capability that helps occupational health and compliance teams translate OSHA-driven medical surveillance requirements into electronic clinical quality measures (eCQMs) written in Clinical Quality Language (CQL), and then run those measures against employee health records to identify who is compliant, due soon, or overdue.

Occupational health programs are accountable for complex requirements such as annual physicals, respirator clearance, TB screening, immunizations, and role-based surveillance. Yet, the work is often managed through spreadsheets, manual chart review, and ad hoc reporting. WorkWell Measure Studio standardizes this process by pairing an AI-assisted measure authoring workflow with explicit, testable rule logic and a case-oriented worklist that drives follow-up actions.

Key capabilities include:

- **OSHA-to-eCQM measure authoring:** Convert OSHA regulations and internal policies into versioned eCQM definitions, including required data elements, eligibility criteria, exclusions, and compliance windows.
- **CQL execution against occupational health data:** Load CQL measures into a sandbox or connected environment and evaluate them against structured employee health data to produce clear compliance outcomes (compliant, due, overdue) and cohorts.
- **Evidence-based explanations for each result:** For every flagged employee, the system returns the measure logic path and the underlying data points used, for example, role, last completed exam date, expiration window. This way, teams can trust and audit the output.
- **Case Worklist built for compliance operations:** Replace passive dashboards with an operational worklist that groups non-compliant employees into actionable queues, supports filtering by policy/role/location and drives consistent outreach and scheduling workflows.
- **Safe AI integration for productivity:** AI accelerates measure drafting, summarization, and user Q&A, but compliance decisions remain grounded in explicit eCQM/CQL logic and traceable evidence thereby, reducing risk and increasing reliability.

*"Occupational health teams don't need another dashboard. They need a system that turns regulations into executable rules and produces an accountable list of what to do next," said [MIE/Enterprise Health Spokesperson Name], [Title]. "**WorkWell Measure Studio** shortens the*

path from policy to practice, so teams can spend less time interpreting requirements and more time keeping employees safe and compliant."

A customer perspective:

*"We have strong policies, but operationalizing them across thousands of employees is time-consuming," said [Customer Name], [Title], [Company]. "**WorkWell Measure Studio** gives us a consistent way to define surveillance rules once, run them repeatedly, and work a clean queue of exceptions without living in spreadsheets."*

Availability

WorkWell Measure Studio will be available to Enterprise Health clients via [release phase: internal pilot / early access / general availability]. Early access includes a starter library of medical surveillance and OSHA-adjacent measure templates, a synthetic-data sandbox for validation, and guided onboarding for teams who want to operationalize CQL-based compliance checks.

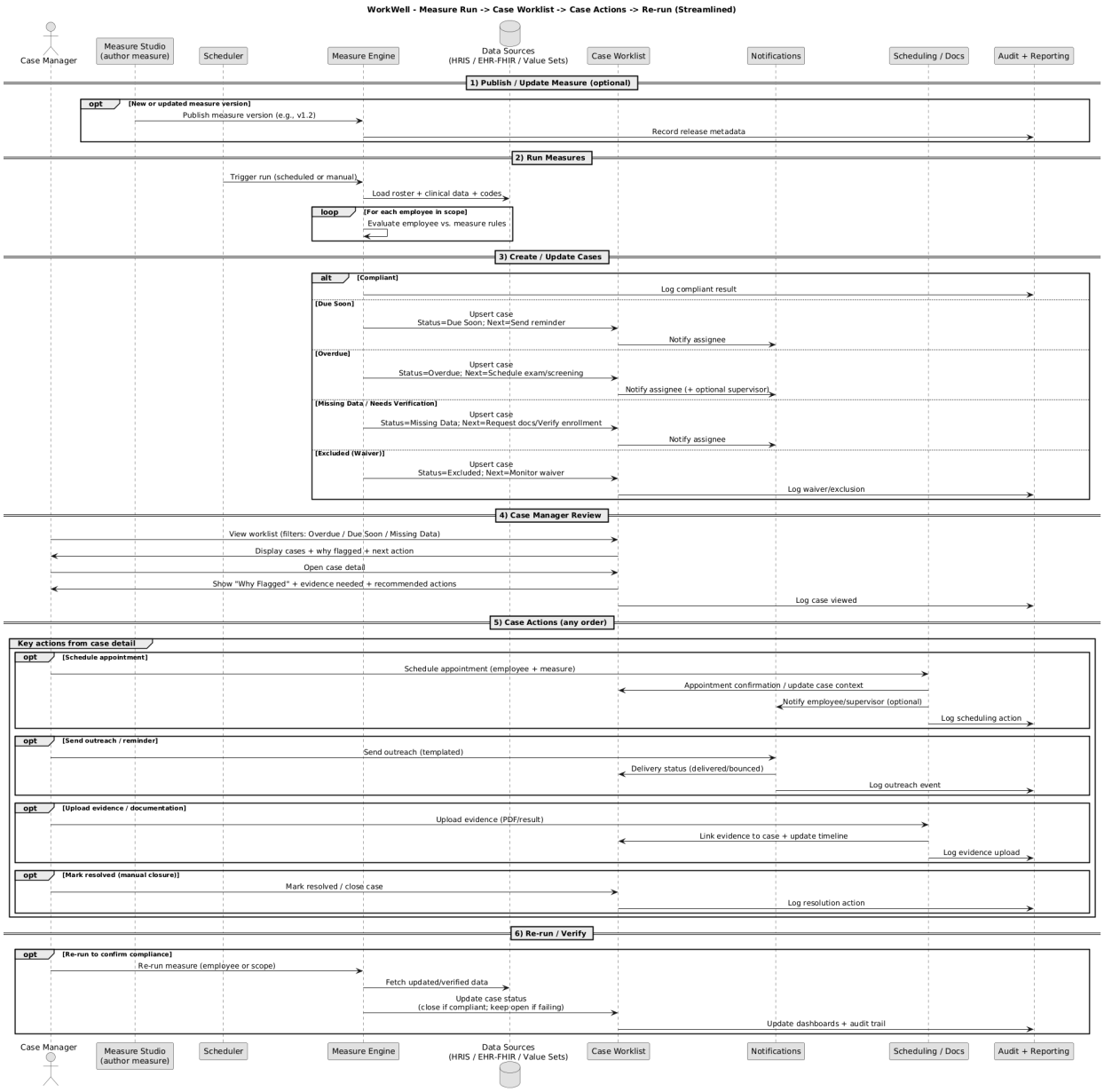
For more information or to schedule a demo, visit https://enterprisehealth.com/

About Enterprise Health

Enterprise Health is a leading provider of occupational health and employee well-being solutions, serving organizations worldwide with innovative, integrated technology.

Media Contact:
Enterprise Health Communications
info@enterprisehealth.com
https://enterprisehealth.com

Workflow:



Proof of Concept Screenshots:

WW WorkWell Measure Studio

Search employees, measures, cases... | All Sites | Last 30 Days

- Programs
- Worklist
- Measures
- Measure Studio
- Test Runs
- Admin

Programs Overview

Monitor compliance across all occupational health programs

Hearing Conservation

v1.2 +2.3%

OSHA 29 CFR 1910.95

Annual Audiogram Completed

156	23	12	5
Compliant	Due Soon	Overdue	Missing Data

Compliance Rate: 80% | Last run: 2026-01-05 14:32

View Details | Worklist

HAZWOPER Medical Surveillance

v1.0 -1.1%

OSHA 29 CFR 1910.120

Annual Medical Surveillance Exam

89	15	8	3
Compliant	Due Soon	Overdue	Missing Data

Compliance Rate: 77% | Last run: 2026-01-05 08:15

View Details | Worklist

TB Surveillance

v1.3 +0.5%

Org Policy / CDC

Annual TB Screening by role

142	18	14	7
Compliant	Due Soon	Overdue	Missing Data

Compliance Rate: 78% | Last run: 2026-01-05 11:20

View Details | Worklist

Immunization Compliance

v2.1 +4.2%

Org Policy / CDC

Flu Vaccine this season by role

178	12	6	9
Compliant	Due Soon	Overdue	Missing Data

Compliance Rate: 87% | Last run: 2026-01-04 16:45

View Details | Worklist

WW WorkWell Measure Studio

Search employees, measures, cases... | All Sites | Last 30 Days

- Programs
- Worklist
- Measures
- Measure Studio
- Test Runs
- Admin

← Back to Programs

TB Surveillance

Active v1.3

Open Worklist (Filtered)

Org Policy / CDC — Employees in high-risk roles must complete annual TB screening per CDC guidelines.

Compliant	Due Soon	Overdue	Missing Data
142	18	14	7

Compliance Trend

Aug Sep Oct Nov Dec Jan

78.5%

Current compliance rate

+2.5% from Aug

Top Drivers (Non-Compliance)

BY SITE

Clinic — 9 overdue — High patient volume

BY ROLE

Nurse — 7 overdue — Night shift coverage

BY REASON CODE

MISS_ANNUAL — 11 cases (79%)

WAIVED — 3 cases (21%)

Measures in this Program

Measure	Version	Status	Compliant	Due Soon	Overdue	Actions
Annual TB Screening by role	v1.3	Active	142	18	14	Edit

WW WorkWell
Measure Studio

- Programs
- Worklist
- Measures
- Measure Studio
- Test Runs
- Admin

Search employees, measures, cases...

All Sites | Last 30 Days

Measure Catalog

Browse and manage compliance measures

+ Create Measure

All Measures

Measure Name	OSHA/Policy Ref	Version	Status	Owner	Last Updated	Tags	Actions
Annual Audiogram Completed	OSHA 29 CFR 1910.95	v1.2	Active	J. Chen	2025-11-12	Surveillance, Exam, Hearing	⋮
Annual Medical Surveillance Exam	OSHA 29 CFR 1910.120	v1.0	Active	M. Patel	2025-08-20	Surveillance, HAZWOPER	⋮
Annual TB Screening by role	Org Policy / CDC	v1.3	Active	M. Patel	2025-10-18	Surveillance, TB	⋮
Flu Vaccine this season by role	Org Policy / CDC	v2.1	Active	K. Williams	2025-09-30	Vaccine, Seasonal	⋮
Baseline Audiogram (New Hire)	OSHA 29 CFR 1910.95	v1.0	Active	J. Chen	2025-06-15	Surveillance, Onboarding	⋮
Respiratory Fit Test	OSHA 29 CFR 1910.134	v0.9	Draft	J. Chen	2026-01-04	Surveillance, Respiratory	⋮
Hepatitis B Vaccination Series	OSHA 29 CFR 1910.1030	v2.0	Approved	K. Williams	2025-12-01	Vaccine, BBP	⋮
Lead Medical Surveillance	OSHA 29 CFR 1910.1025	v1.1	Deprecated	M. Patel	2024-03-15	Surveillance, Lead	⋮

Search employees, measures, cases...

All Sites

Last 30 Days

Measure Studio

Author and test compliance measures

Annual Audiogram Completed v1.2 Draft

OSHA 29 CFR 1910.95

v1.1 → v1.2 | Save Draft

All tests passing.
This measure is ready for approval review.

Spec | CQL | Value Sets | Tests | Release/Approval

Measure Name

Annual Audiogram Completed

OSHA/Policy Reference

OSHA 29 CFR 1910.95 - Occupational Noise Exposure

Description

Employees in hearing conservation program must complete annual audiogram

Paste Requirement Text (Optional)

Paste OSHA regulation text or policy requirement here ...

AI Draft Spec

Eligibility Criteria

Role Filter

Maintenance Tech, Welder, Industrial Hygienist

Site Filter

Plant A, Plant B

Program Enrollment

Spec | CQL | Value Sets | Tests | Release/Approval

CQL Logic (FHIR R4)

Compiled 0 warnings

```
library AnnualAudiogramCompleted version '1.2'

using FHIR version '4.0.1'

include FHIRHelpers version '4.0.1' called FHIRHelpers

codesystem "LOINC": 'http://loinc.org'
codesystem "SNOMED": 'http://snomed.info/sct'

valueset "Audiogram Procedures": 'urn:oid:2.16.840.1.113883.3.464.1003.118.11.1'
valueset "Medical Waiver Conditions": 'urn:workwell:vs:medical-waiver'
valueset "Hearing Conservation Enrollment": 'urn:workwell:vs:hearing-enrollment'

code "Audiometry study": '28615-2' from "LOINC"
code "Medical exemption": '183932001' from "SNOMED"

parameter "Measurement Period" Interval<DateTime>

context Patient

// Eligibility check
define "In Hearing Conservation Program":
  exists([Condition: "Hearing Conservation Enrollment"])
  or exists([Observation] O where O.code.coding.display ~ 'Noise Exposure ≥ 85 dBA')

// Waiver detection: Condition-based OR DocumentReference-based
define "Has Active Waiver":
  exists(
    [Condition: "Medical Waiver Conditions"] C
    where C.clinicalStatus ~ 'active'
      and (C.abatement is null or C.abatement after Today())
  )
  or exists(
    [Condition] C
    where C.code.coding.code = '183932001'
      and C.clinicalStatus ~ 'active'
  )
  or exists(
    [DocumentReference] D
```

Required Data Elements

- Procedure (Audiogram)
- Condition (Enrollment, Waiver)
- Observation (Noise Exposure)
- DocumentReference (Waiver docs)

Dependencies

- FHIRHelpers v4.0.1
- LOINC CodeSystem
- SNOMED CT CodeSystem

Status Logic

- **Excluded**: Active waiver (highest priority)
- **Compliant**: ≤335 days since last
- **Due Soon**: 336-365 days
- **Overdue**: >365 days
- **Missing Data**: No records

Test Runs

Measure execution history and results

Run All Measures Now

Run History

Timestamp	Run ID	Scope	Site	Employees	Pass Rate	Failures	Duration	Trigger	Logs
2026-01-05 14:32:01	RUN-2026-01-05-143201	All Programs	All Sites	565	64.6%	12	4m 32s	Scheduled	
2026-01-05 08:15:00	RUN-2026-01-05-081500	HAZWOPER Medical Surveillance	Plant A	115	77.4%	3	1m 15s	Manual	
2026-01-04 16:45:00	RUN-2026-01-04-164500	Immunization Compliance	All Sites	205	86.8%	2	2m 08s	Scheduled	
2026-01-04 11:20:00	RUN-2026-01-04-112000	TB Surveillance	Clinic	181	78.5%	5	1m 45s	Manual	
2026-01-03 14:30:00	RUN-2026-01-03-143000	Hearing Conservation	Plant A, Plant B	196	79.6%	4	2m 22s	Scheduled	
2026-01-02 09:00:00	RUN-2026-01-02-090000	All Programs	All Sites	562	63.7%	15	4m 48s	Scheduled	

Admin

System configuration, integrations, and governance

Integrations

Connected systems and data feeds

SF	SAP SuccessFactors (HRIS) Employee roster, roles, sites, supervisors	Connected
EHR	Epic (EHR) Procedures, encounters, lab results (FHIR R4)	Connected
IH	Industrial Hygiene DB Exposure assessments, program enrollments	Connected
LMS	Learning Management System Training completions	Not Connected · Connect

Data Feed Status

Last sync times and health

HRIS Employee Roster Daily at 02:00 UTC	Healthy	Last: 2026-01-06 02:00
EHR Clinical Data Real-time (webhook)	Healthy	Last: 2026-01-06 10:45
IH Exposure Data Weekly on Monday	Delayed	Last: 2025-12-30 06:00

Trigger Manual Sync

SSO & Role Mapping

Identity provider and SCIM configuration

Azure AD (Entra ID)
SAML 2.0 · SCIM provisioning enabled — Active

Role Mappings

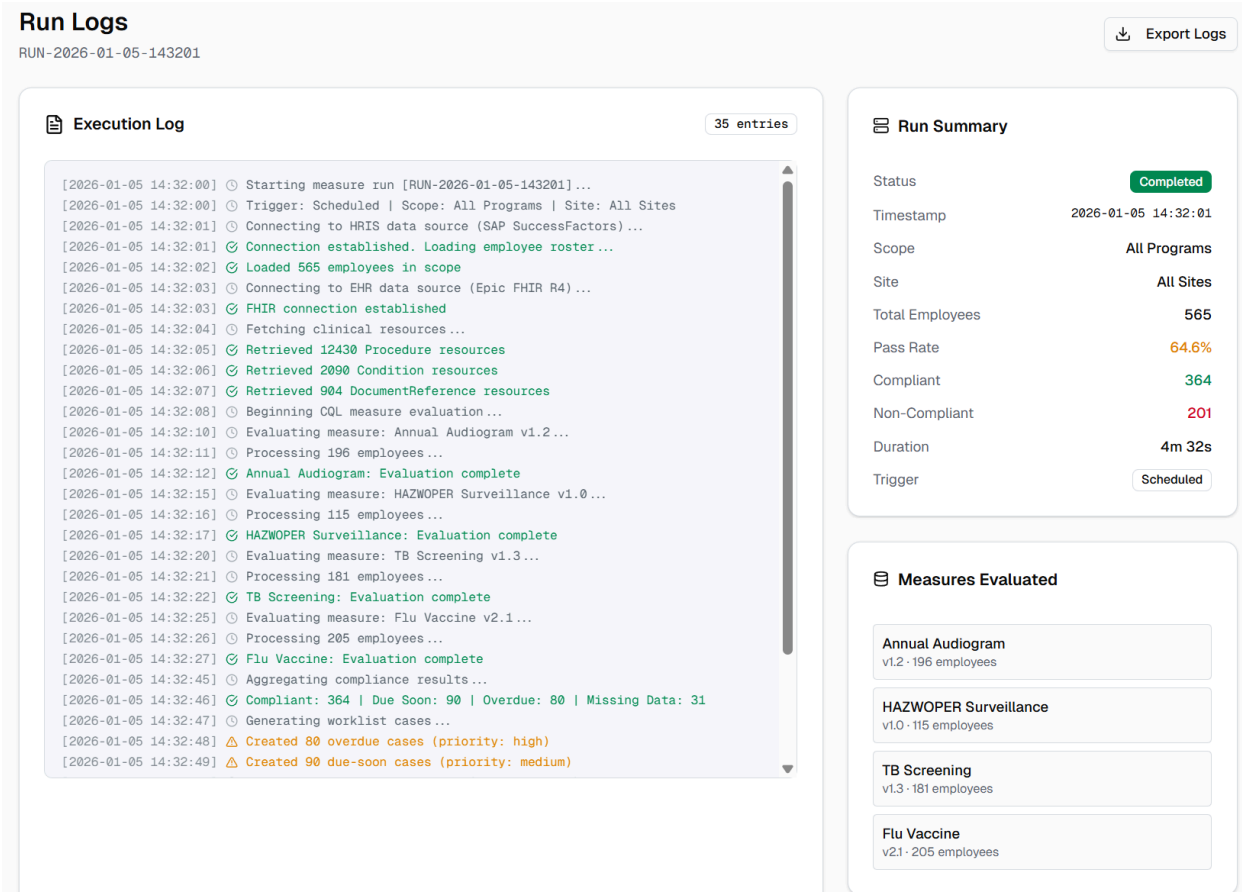
AD Group: OHS_Admins	→ Admin
AD Group: OHS_Analysts	→ Compliance Analyst
AD Group: OHS_Viewers	→ Read-only

Edit Role Mappings

Notification Templates

Outreach and reminder configurations

Due Soon Reminder Email · 30 days before due	Edit
Overdue Notice Email · On overdue date	Edit
Supervisor Escalation Email · 14 days after overdue	Edit



epic: WorkWell Measure Studio MVP (policy-to-CQL authoring → measure runs → actionable worklist)

Title

WorkWell Measure Studio MVP: Author OSHA/policy measures as eCQMs (CQL) and operationalize compliance with runs, worklists, and case actions

Executive Summary

Occupational health and compliance teams need a repeatable way to translate OSHA-driven surveillance requirements into executable, testable rules and then operationalize the results as an accountable queue of follow-up work. This epic delivers a cohesive MVP that lets a measure author define a versioned measure (spec + value sets + CQL + tests), approve/release it, run it on a schedule or manually against connected data sources (HRIS + EHR/FHIR + IH exposure), and automatically generate/update cases in a worklist for case managers to take action

(schedule, outreach, upload evidence, resolve). The system must be auditable: every flagged outcome must be explainable via “why flagged” evidence and a traceable audit log.

User Story

As an occupational health compliance team, we want to define surveillance requirements once as versioned measures, run them repeatedly across our employee population, and work an actionable queue of non-compliance cases with clear evidence and standardized follow-up actions.

Success Criteria (MVP)

- A measure author can create a measure, compile CQL, attach value sets, run tests, and release a version to Active.
- A scheduled or manual run produces a run summary and per-employee outcomes for all measures in scope.
- Non-compliant outcomes upsert cases into a worklist with an assignee, status, priority, and next action.
- A case manager can open a case, see why flagged + evidence timeline, take at least one action, and re-run to verify closure.
- Every run, case update, and notification is logged in an audit trail.

Measurable Outcomes (MVP)

- Measure authoring lead time: draft → Active in under 1 business day for a single measure (excluding stakeholder review).

- Run reliability: at least 95% successful runs in the demo environment, failures produce actionable error summaries.

- Explainability coverage: at least 95% of flagged outcomes display “Why Flagged” evidence with the key data elements used.

- Operational integrity: duplicate-case rate is near zero across repeated runs (idempotent case upsert behavior).

- Closure loop: a case manager can close and verify a case via rerun in one pass for the primary demo measure.

Tradeoffs and Key Decisions (MVP)

- CQL is the source of truth for compliance logic, the Spec tab is structured metadata used for auditability, UI explainability, and authoring clarity (not a second rules engine).

- “Compile/Validate” is a backend service call to keep the editor lightweight and to standardize validation output (errors/warnings) across clients.

- Runs are designed to be resumable and auditable, the system persists run summaries and outcome artifacts before generating/updating worklist cases.

- Cases are created/updated via deterministic “upsert rules” (employee + measure version + evaluation period) to prevent duplicates and enable rerun verification.

- The MVP supports a demo-grade connected flow with clear fallbacks (stubs/simulated connectors) rather than full environment hardening.

Risks and Mitigations (MVP)

- Data quality gaps (missing roster fields, incomplete clinical records) may cause false positives. Mitigation: explicit “Missing Data” outcome state and required data element surfacing; do not treat missing data as overdue.

- Value set drift or code mismatches. Mitigation: show referenced value sets and dependencies; include compile-time validation and basic runtime checks.

- Over-alerting creates worklist fatigue. Mitigation: prioritize and categorize outcomes (Due Soon vs Overdue vs Missing Data vs Excluded), provide filters/queues, and support templated notifications.

- Long-running or partially failed runs. Mitigation: persist run logs and partial progress; show retriable failures and keep prior outcomes/cases stable until replacement outcomes are confirmed.

- Audit expectations. Mitigation: audit events for releases, runs, notifications, and case lifecycle transitions; link audit entries to run IDs and case IDs.

Out of Scope (for MVP)

- Full production-grade integration hardening for every customer environment (focus on a demo-grade “connected” flow + simulated fallbacks).
- Complex multi-step approvals across many orgs, MVP is a single approval chain.
- Advanced analytics beyond baseline dashboards and exportable run logs.

- Full customization of case routing rules across departments beyond basic assignment and escalation.

Deliverables
- Measure Studio: catalog + authoring UI (spec/CQL/value sets/tests/release)
- Measure Engine: scheduled/manual execution with run logs + run summaries
- Worklist + Case Detail: queues, filters, evidence, actions, re-run verification
- Admin: integrations status + role mapping + notification templates (minimum viable)
- Audit + reporting hooks

Sequencing Plan (MVP)
- Phase 1: Authoring backbone (Measure Catalog + Spec + CQL compile) and one end-to-end demo measure authored and releasable.

- Phase 2: Execution backbone (manual run + run summary/logs + persisted outcomes) with deterministic rerun behavior.

- Phase 3: Operations backbone (worklist case upsert + case detail + actions + rerun verification) with complete audit trail.

Sample ticket set (refactored from the epic into implementable issues)

Ticket 1: Measure Catalog + Versioning (CRUD + statuses)

Executive Summary
Provide a catalog view and backing model for measures that supports versioning (v1.1 → v1.2), ownership, policy references, and lifecycle states (Draft, Approved, Active, Deprecated). This becomes the navigation backbone for Measure Studio.

User Story
As a measure author, I want to browse all measures, see their status/version, and quickly create or update a measure version.

Acceptance Criteria
- Catalog table lists: Measure Name, OSHA/Policy Ref, Version, Status, Owner, Last Updated, Tags.
- “Create Measure” flow creates Draft v0.x or v1.0 (configurable).
- “New Version” clones prior version into a new Draft (e.g., v1.1 → v1.2) with a change summary field.
- Lifecycle states enforced: Draft → Approved → Active; Deprecated is terminal.

- Non-goal (MVP): bulk import/export of measures.

Notes / Dependencies
- Depends on basic auth/roles (Author vs Viewer).
- Align fields with UI screenshot: Owner, Tags, Last Updated, Status pill.

Ticket 2: Measure Studio Spec Tab (eligibility, exclusions, policy refs)

Executive Summary
Implement the structured “Spec” authoring surface that captures the measure definition separate from CQL: required data elements, eligibility criteria, exclusions, compliance window, and policy references.

User Story
As a measure author, I want to define the human-readable and structured requirements for the measure so it is auditable and can drive UI explanations.

Acceptance Criteria
- Spec fields: name, description, policy reference, program enrollment criteria, role filter, site filter.
- A “Required Data Elements” panel exists (can be auto-derived from CQL later; MVP can be manual with optional auto-fill).
- Exclusions section supports “waiver/exemption” definition (label + criteria).
- Save Draft persists without requiring compiled CQL.
- Non-goal (MVP): fully auto-deriving all spec fields from CQL.

Example Scenario
- Annual Audiogram Completed: role filter includes Maintenance Tech/Welder/IH; site filter Plant A/B; policy OSHA 29 CFR 1910.95.

Notes / Dependencies
- Should be compatible with “AI Draft Spec” button (Ticket 6) (not required for MVP).

Ticket 3: CQL Editor + Compile/Validate (FHIR R4)

Executive Summary
Add an embedded CQL editor that supports compile/validate, shows warnings, and displays dependencies (FHIRHelpers, LOINC, SNOMED) and referenced value sets. This is the “executable logic” source of truth.

User Story
As a measure author, I want to write CQL and confirm it compiles so runs are trustworthy.

Acceptance Criteria
- CQL tab provides editor, compile button, compile status (Compiled, Warnings count, Errors list).
- Display dependencies list (FHIRHelpers version, CodeSystems used).
- Display referenced ValueSets (by name + identifier).
- Block Release/Approval if compile fails.
- Non-goal (MVP): advanced editor features (autocomplete, linting, refactors).

Notes / Dependencies
- Pairs with Ticket 4 (Value Sets) and Ticket 5 (Tests).
- MVP can treat compile as a backend service call returning status/warnings.

Ticket 4: Manual Measure Run Orchestrator + Run Logs + Run Summary

Executive Summary
Implement a “Run Measures Now” capability that evaluates all Active measures (or a selected scope) against connected data sources and persists a run summary, run logs, and per-employee outcomes.

User Story
As a compliance analyst, I want to trigger a run and see what happened (summary + logs) so I can trust the results and take action.

Acceptance Criteria

- A manual run can be initiated for a scope (All Programs / selected program / selected measure).
- The system creates a Run record (Run ID, timestamp, scope, site, trigger type) and appends execution logs.
- Run Summary includes: total employees evaluated, pass rate, compliant count, non-compliant count, duration.
- Per-employee outcomes are persisted with outcome status (Compliant / Due Soon / Overdue / Missing Data / Excluded) and a pointer to “Why Flagged” evidence.

- Non-goal (MVP): full scheduling UI; manual trigger is sufficient (scheduler can be simulated).

Notes / Dependencies

- Depends on data access layer to HRIS/EHR/IH (demo-grade connectors acceptable).
- Requires outcome storage schema that supports downstream case upserts.

Ticket 5: Worklist Case Upsert Engine (cases from outcomes, idempotent reruns)

Executive Summary
Automatically create/update worklist cases from non-compliant outcomes using deterministic upsert rules to prevent duplicates and to support rerun verification.

User Story
As a case manager, I want non-compliant employees to appear as cases in my worklist with a clear status and next action so I can work the queue consistently.

Acceptance Criteria

- For each non-compliant outcome (Due Soon / Overdue / Missing Data), the system upserts a case keyed by (employee, measure version, evaluation period).
- Cases include: assignee, status, priority, next action, created/updated timestamps, and link to last run ID.
- Excluded outcomes do not generate active cases, they are recorded as “Excluded” with waiver/exclusion context.
- Rerun behavior: if an employee becomes Compliant, the case is closed/resolved automatically (or marked “Ready to close” if manual closure is required, choose one for MVP and document it).
- Non-goal (MVP): complex routing rules, default assignment + optional supervisor escalation is sufficient.

Notes / Dependencies

- Depends on Ticket 4 outcome persistence.

- Requires a minimal case status model and priority mapping (e.g., Overdue = High, Due Soon = Medium, Missing Data = Medium).

Ticket 6: Case Detail Evidence ("Why Flagged") + One Action + Rerun-to-Verify + Audit Events

Executive Summary
Deliver the operational case experience: explainability ("why flagged"), evidence timeline, at least one actionable workflow, and rerun verification. Ensure audit events cover all key actions.

User Story
As a case manager, I want to open a case, understand why it exists, take action, and verify closure with a rerun, with a complete audit trail.

Acceptance Criteria

- Case Detail shows: employee + measure context, status/priority/next action, and a "Why Flagged" section listing the key data elements used (e.g., last exam date, role/site eligibility, waiver status).

- Evidence timeline supports at least: (a) outreach note OR (b) evidence upload metadata OR (c) scheduling confirmation (pick at least one for MVP).

- Rerun-to-verify: from the case detail, trigger a rerun for the employee/measure and update case state based on the new outcome.

- Audit trail records: run initiated/completed, case created/updated/closed, outreach sent, evidence uploaded, scheduling action taken.

- Non-goal (MVP): full document storage, evidence can be metadata + link placeholder.

Notes / Dependencies

- Requires a minimal audit event schema and consistent linking (Run ID ↔ Case ID ↔ Measure Version).

- Works best when "Why Flagged" evidence is stored as structured fields rather than only free text.

Optional: Remaining MVP ticket map (titles only, to show full decomposition without clutter)

- Value Sets Manager (attach/search/import value sets, show identifiers)
- Tests Tab (define test fixtures + expected outcomes, block release if failing)
- Approval/Release Flow (single approver chain + version activation)
- Notifications (templates + send on case creation/escalation, delivery status)
- Admin Integrations Health (connector status, last sync, manual sync trigger)
- Baseline Reporting/Export (CSV export of run summary + outcomes + case states)

Execution plan (sequencing and milestones)

Phase 0: foundation
Objective: establish the minimum skeleton so every later feature has a real home.
Deliverables:

- roles/permissions: Author, Approver, Case Manager, Admin
- measure data model: Measure, MeasureVersion, Status, Ownership, PolicyRef, Tags
- audit log primitives: append-only events for measure edits, releases, runs, case updates

Phase 1: authoring MVP loop
Objective: a measure can be authored end-to-end in a “dry” environment (compile + validate + test), even before real data is connected.
Deliverables:

- Measure Catalog + Versioning (Ticket 1)
- Spec Tab (Ticket 2)
- CQL Editor + Compile/Validate (Ticket 3)

- minimal “release gate”: cannot approve/release unless compile passes

Phase 2: run pipeline + outcomes
Objective: run an Active measure and produce per-employee outcomes with explainability.
Deliverables:

- manual run trigger + scheduled run stub (cron or job runner)
- run summary (counts, duration, data freshness, errors)
- per-employee outcome record with “why flagged” evidence payload (data elements + rule path)

Phase 3: worklist + case actions
Objective: non-compliance becomes operational work with accountability.
Deliverables:

- worklist queue (filters: status, priority, assignee, measure, site)
- case detail (evidence timeline + actions)
- at least one action type end-to-end (example: schedule follow-up or request document)
- re-run verification path to confirm closure

Phase 4: demo hardening
Objective: a clean, repeatable demo with credible auditability.
Deliverables:

- seeded demo dataset (HRIS + FHIR + exposure minimal fields)
- deterministic test cases proving one compliant and one non-compliant scenario
- exportable run log and audit trail view

Systems awareness (what components exist in MVP)

- Measure Studio (UI): catalog, versioning, spec authoring, CQL authoring, release
- Compile service: validates CQL + resolves dependencies + returns warnings/errors
- Run service: executes Active measures on a schedule/manual trigger and writes outcomes
- Case service: upserts cases from outcomes and tracks lifecycle/actions
- Audit/event log: immutable stream of key actions (measure changes, releases, runs, case edits)
- Integrations (MVP-level): one “connected” path plus simulated fallback data for demo reliability

Risks and mitigations

Risk: data incompleteness or mismatched coding systems causes false flags
Mitigation: require “data freshness + missingness” indicators in run summary; show “insufficient data” vs “non-compliant” as distinct outcomes.

Risk: value sets are hard to manage and lead to brittle measures
Mitigation: ship a minimal value set registry (IDs + names + versions) and require referenced value sets to be resolvable at compile/release time.

Risk: case volume overwhelms teams (too many flags)
Mitigation: include priority rules in outcome-to-case mapping and allow scoping by site/role/program in Spec.

Risk: auditability is hand-waved
Mitigation: append-only audit events for the five critical actions: measure edit, approval/release, run start/end, case create/update, notification sent.

Remaining backlog (named, not fully ticketed)

- Value Sets Manager: import/register value sets, versioning, linking to measures

- Test Harness: define test fixtures, expected outcomes, and regression tests per version
- Scheduling + Notifications: schedules per measure, alert templates, recipient routing
- Worklist Enhancements: bulk actions, SLA timers, escalations
- Reporting/Export: run history export, case audit export
- Integration Hardening: connectors, retries, mappings, and environment-specific config

Open questions (for reviewer pressure-test)

- what is the minimum FHIR resource set required for the first demo measure (Observation, Condition, Procedure, Encounter)?
- how should “insufficient data” be represented so it does not create compliance noise but remains visible?
- what is the default case lifecycle (New → In Progress → Waiting on Employee → Resolved → Closed) for MVP?
- what is the minimal evidence payload that is both explainable and safe (PII constraints)?
- who owns measure approval in a real deployment (OH director, compliance lead, medical director)?

FAQ:

1. **What problem does WorkWell Measure Studio solve?**
   It converts OSHA/policy surveillance requirements into executable, testable measures (CQL) and operationalizes results as an accountable worklist of follow-up cases (instead of spreadsheets and manual chart review).

2. **What is a “measure” in this MVP?**
   A versioned compliance definition that includes Spec metadata (eligibility, exclusions, compliance window, required data elements) plus executable CQL (FHIR R4) and referenced value sets. A measure version is what gets approved/released and run.

3. **What outcome statuses exist and what do they mean?**
   Compliant (meets requirement), Due Soon (approaching deadline), Overdue (past deadline), Missing Data (cannot evaluate safely), Excluded (waiver/exemption or defined

exclusion). Only Due Soon, Overdue, and Missing Data generate active worklist cases.

4. **How does “why flagged” work?**
   Every non-compliant outcome stores a structured evidence payload (key inputs like role/site eligibility, last qualifying exam date, waiver status, and the compliance window calculation). The case detail shows this evidence and links it to the run/audit trail.

5. **How do we prevent duplicate cases on reruns?**
   Cases are idempotently upserted using a deterministic key (employee + measure version + evaluation period). Reruns update the same case instead of creating new ones.

6. **What is the minimum demo flow for MVP completeness?**
   Author and release one measure → trigger a run → generate outcomes and cases → open a case, take one action (outreach/scheduling/evidence metadata) → rerun to verify and resolve/close → audit log shows the full trail.

V1.0 Proposal

Introduction:

This project proposes an occupational health compliance agent sandbox that uses an AI agent, explicit policy rules, and the Model Context Protocol (MCP) to reason over synthetic employee health data.

The system will simulate Enterprise Health style workflows on a small, realistic dataset and a limited set of occupational health policies (e.g., annual physicals, TB screenings, vaccinations).

The agent will expose its capabilities through MCP tools so it can be called by AI assistants and a simple web UI, demonstrating how modern agentic workflows can help administrators quickly identify non-compliant employees and suggest follow-up actions.

Context and Prior Summer 2025 Work:

In my Summer 2025 internship, I worked on a medical document RAG system that:

- Ingested and indexed documents using OpenSearch and embedding models.
- Implemented chunking and retrieval pipelines, experimented with different embedding configurations, and began exploring RAG evaluation frameworks.
- Introduced an MCP client/server and started an npm package to make RAG functionality accessible as tools.
- Built a Ruby on Rails plugin to integrate the RASS engine's semantic search into Redmine, improving workflow and demonstrating how AI services can be embedded into existing applications.

That project produced a solid foundation for semantic search and MCP integration. However, it stayed focused on document retrieval and Redmine integration rather than moving up a level to model real occupational health workflows. This proposal is a continuation in a new direction by using the earlier MCP and RAG infrastructure experience to design a small but realistic compliance assistant that reasons over structured employee health data and policy rules, in a sandbox that mirrors Enterprise Health style use cases without requiring production system access.

Problem and Motivation:

Occupational health programs must ensure that employees complete required exams, vaccinations, and screenings based on their job roles and risk profiles. In practice, this often means:

- Manually tracking due and overdue items across many employees and policies.
- Interpreting policy rules (e.g., "annual physical for these roles," "TB screening for these roles") and mapping them to each employee's record.
- Generating reminders, lists, or tasks to close gaps in compliance.

Modern AI systems increasingly use tools using agents and protocols like MCP to connect large language models with structured data, rules engines, and external actions. While this is promising for occupational health, it is not yet clear what a practical, scoped compliance agent should look like, how its tools should be designed, and how it can be safely integrated into existing products.

A lightweight, realistic, and multitasking environment can help:

- Model a subset of occupational health policies and employee data.
- Allow an MCP-based agent to call tools that check compliance and summarize gaps.

It can also help demonstrate how such an agent could be surfaced in a UI and eventually integrated into a system like Enterprise Health, without requiring direct access to production data.

Project Goals:

Agent Sandbox

Build a sandbox environment that mimics key aspects of an occupational health system:

- Create a synthetic dataset of employees with fields such as job role, last physical exam date, last TB screening date, and vaccination dates.
- Encode a small set (e.g., 3–5) of realistic occupational health policies (annual physicals, TB screenings, flu shots, etc.) as explicit, machine-readable rules.
- Implement logic to determine, for a given employee, whether each applicable policy is compliant, due, or overdue, and what follow-up actions are recommended.

Compliance Evaluation and Summarization

Design tools and APIs to allow the agent and UI to query compliance status at both the individual and cohort level:

- Functions to check compliance for a single employee, returning structured information about violated or upcoming rules and suggested actions.

- Functions to list non-compliant employees for a given policy and to produce simple aggregate summaries (e.g., “10 employees overdue for TB screening”).

MCP Integration

Wrap the sandbox’s capabilities in an MCP server so an AI assistant can orchestrate them:

- Expose tools such as `get_employee`, `check_compliance`, `list_noncompliant`, and `explain_rule` via MCP.

- Demonstrate how an agent can interpret natural language questions (“Who is overdue on TB screening?”) into appropriate MCP tool calls and present the results in a clear format.

Reusable Toolkit and UI

Deliver code, configuration, and documentation that make the sandbox reusable and easy to understand:

- A small web dashboard that displays employees and their compliance status, with drill-down into which rules apply and what actions are recommended.

- Documentation explaining how the sandbox data model, rules, and tools map onto a potential Enterprise Health or WebChart integration, and how the same patterns could be extended to real systems.

Technology Stack:

- **Language and Frameworks:**
  - Backend: Python (FastAPI/Flask) or Node.js (Express) for the sandbox API and MCP server.
  - Frontend: React (or Vite + React) for a simple compliance dashboard and optional chat-style interface.

- **Data and Rules:**
  - SQLite or PostgreSQL for storing synthetic employee records.
  - JSON/YAML-based rule definitions for occupational health policies

    - Optional use of a lightweight rules engine or graph library for rule evaluation.

- **Models and Agent:**
    - A hosted large language model (e.g., OpenAI, Gemini, or similar) for natural language understanding and response generation.
    - Agent logic that focuses on orchestrating MCP tool calls rather than embedding business rules inside the model.

- **MCP:**
    - MCP server implementation in Python or Node.js, building directly on my prior MCP client/server and npm work to define clear and typed tools for compliance queries.

Expected Outcomes:

Overall the project aims to deliver:

- A working occupational health compliance sandbox with synthetic employee data and a small set of encoded policies.

- An MCP-compatible set of tools (e.g., `check_compliance`, `list_noncompliant`) that an AI agent can call to reason about compliance and recommend actions.

- A simple web UI that shows individual and cohort-level compliance status and surfaces agent explanations in an understandable way.

- A reusable, documented codebase and design description showing how this sandbox architecture could be mapped onto Enterprise Health style data and workflows, extending my prior RAG and MCP work into a more agentic and workflow-focused direction.